**Experiment No: 05**

**Title:** Implement Stack and Queue using arrays.

**Aim:**

Implement stack, using array and linked list and perform operations like PUSH, POP, and DISPLAY contents of the stack. Also display position of TOP.

**Theory:**

**Stack:**

A stack is an ordered collection of items into which items may be inserted and deleted from one end called Top of stack. The stack is a LIFO i.e. element which is put in the stack the last is the first to come out.

**Static Implementation**:

Since the stack is a ordered list of items we can implement stack using array. The static implementation using array will have limitations on size of stack as array size has to be predefined also the element is not physically deleted so that it does not release memory allotted for the element which has to be removed.

If TOP==-1 it indicates stack is empty.

If TOP==MAX-1 it indicates stack is full.

We shall also define the new operation called 'init' due to which stack will initialise the TOP to -1 to indicate an empty stack.

**Dynamic implementation of stack:**

Dynamic implementation of stack involves stack using Linked list. In

linked list:There is no limitation of size, memory being only constraint.

i) Element will be physically deleted thus releasing the memory allotted for the element which has to be removed.]

ii) There is no leakage of memory.

iii) Since random access to any element is not required in a stack, the linked list representation is preferred over the sequential organisation.

In this implementation the stack will be maintained as a singly linked list.

The first node will be pointed to by a pointer called 'top'. Initially the stack is empty. Hence

there will be no list and top contain NULL. Whenever an element has to be pushed, we will have to make a node to store the element and add this node to the beginning of list. Top will always point to recently added node. When the pop operation has to be performed, the first node will be deleted and the next node will become the new TOP.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 5

int stack[MAX],top=-1;

struct node{
   int data;
   struct node *next;
};

struct node *head=NULL;

void pushArray(int x){
   if(top==MAX-1)
      printf("Stack Overflow\n");
   else
      stack[++top]=x;
}

void popArray(){
   if(top==-1)
      printf("Stack Underflow\n");
   else
      printf("Popped: %d\n",stack[top--]);
}

void displayArray(){
   int i;
   for(i=top;i>=0;i--)
      printf("%d ",stack[i]);
   printf("\nTOP = %d\n",top);
}

void pushList(int x){
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));
```

```
    newnode->data=x;
    newnode->next=head;
    head=newnode;
}

void popList(){
    struct node *temp;
    if(head==NULL)
        printf("Stack Underflow\n");
    else{
        temp=head;
        printf("Popped: %d\n",head->data);
        head=head->next;
        free(temp);
    }
}

void displayList(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}

int main(){
    pushArray(10);
    pushArray(20);
    pushArray(30);
    printf("Array Stack: ");
    displayArray();
    popArray();
    displayArray();

    pushList(10);
    pushList(20);
    pushList(30);
    printf("\nLinked List Stack: ");
    displayList();
    popList();
    displayList();

    return 0;
}
```

Output:

```
Array Stack: 30 20 10
TOP = 2
Popped: 30
20 10
TOP = 1

Linked List Stack: 30 20 10
Popped: 30
20 10
```

M.M.C. O. E.

Page No: Date:

# Assignment 5

Q1 What is LIFO str of stack?

-> The elements in stack are inserted from top & can be removed from top only. Hence element last inserted is removed first.

Q2 How stack is represented?

-> Stack is represented using array or linked list.

Q3 What are the possible operations with stack?

-> 1) PUSH 2) POP 3) stack Empty 4) stack FULL

4 Explain the stack full & stack empty condn.

-> When stack is full top value is equal to -1 & when it is empty top value is 0.

5 Explain push() fn in C.

```
-> void push(stack *S, int x){
   S -> data = S -> top + 1;
   S -> data [S -> top] = x
   }
```

Q6 explain pop().

```
-> int pop(stack *S){
   int x;
   x = S -> data [S -> top];
   S -> top = S -> top - 1;
   return(x);
```

Q7 explain double pointer.

-> pointer to pointer.